CompSci 390-03-26S

Distributed and Networked Systems

Google File System

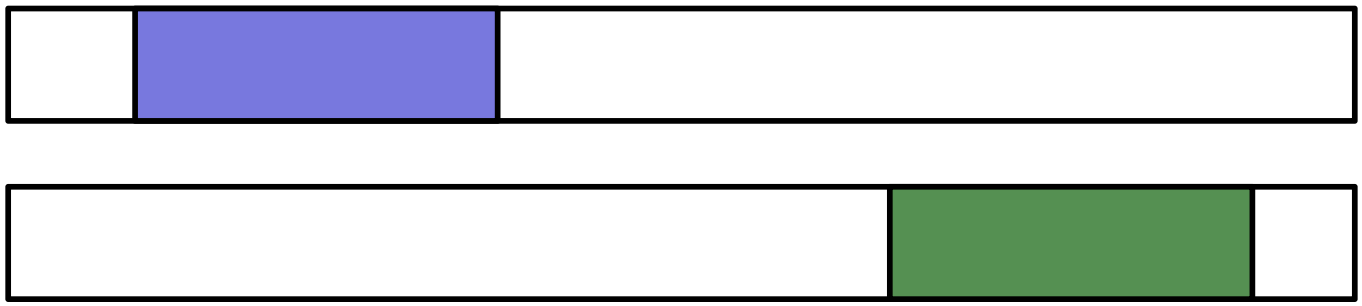
Google File System (GFS)

- Google in 1999: We need a scalable cluster file system: buy or build? Can we do it on the cheap on a collection of commodity servers running Linux?
  - Early research paper from Google: SOSP 2003.
  - Survives after 25 years as Colossus (with some mods).
- Foundation for data-intensive parallel cluster computing at Google
  - MapReduce OSDI 2004
- Because Google developed the clients too, they could rearchitect the file system abstraction to meet their needs.
  - Client access by RPC library
- Inspired various open-source efforts incl. supercomputer file systems.
  - Hadoop HDFS is an “open-source GFS”.

File abstraction

- A **file** is simply a sequence of m bytes

- Reads and writes may apply to arbitrary byte ranges.



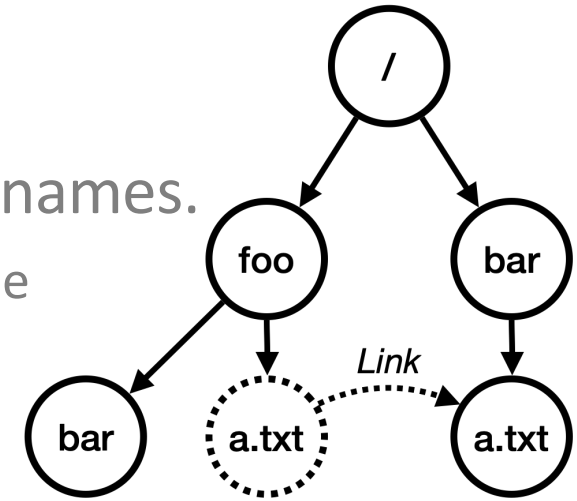
File system API

- Standard operations
  - Create
  - Delete
  - Open
  - Close
  - Read
  - Write
- GFS-specific operations
  - Snapshot – create a copy of a file or directory tree at a point in time, at low cost using copy on write
  - Record append – append a record atomically at least once at an offset of GFS's choosing

The file system tree (or DAG)

Distinguish names (links) from objects ("nodes": files, directories).

- **File**: A named sequence of bytes
- **Directory**: A list of entries—each a name for a file or another directory (a subdirectory). A.k.a **folder**
- **Pathname**: uniquely identifies an object
  - E.g.: */foo/bar* is distinct from */bar*
- **Links**: A Unix/Linux file can have multiple names.
  - **Hard Link**: Another name entry for the same file
  - **Soft Link**: Name to Alternate Name
  - [Links are out of scope for us.]



Files vs. KV stores

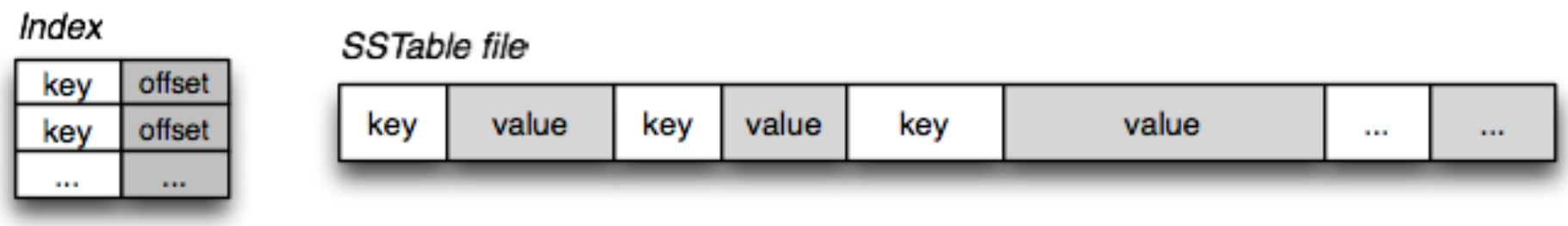
- KV stores have "flat" name spaces: the keys.
  - Easy to scale
  - Not so great for grouping/organizing
- We might think of the key's value as a small file.
  - Read or write in entirety in one put/get op.
  - Key may be a hash of value (**content-addressable**).
- We might store a stream/sequence of KV **records** in a file.

```
type Record struct {
    Key   string
    Value any
}
```

m*sizeof(Record) bytes

Example: Sorted String Tables (SSTable)

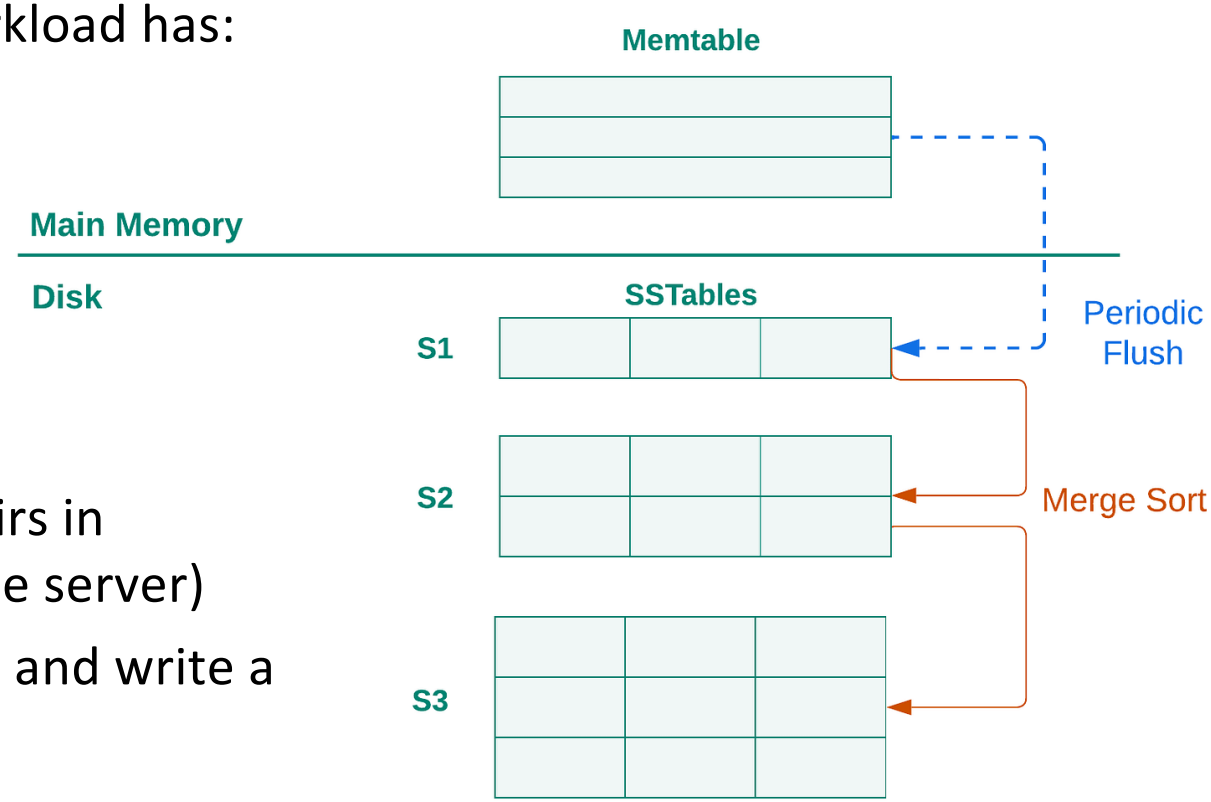
- Google's BigTable and Spanner store KV pairs in sorted GFS files.
- Sorting by key enables range queries: keep close keys together.
- Log/buffer inserted KVpairs in memory, then read/merge/write new file.
- Common technique in modern KV stores.



https://www.igvita.com/2012/02/06/sstable-and-log-structured-storage-leveldb/

Mutable SSTables: log-structured merge tree (LSTM)

- This "common case" workload has:
  - Sequences of records
  - In **Big** files
  - Immutable files
  - Sequential access

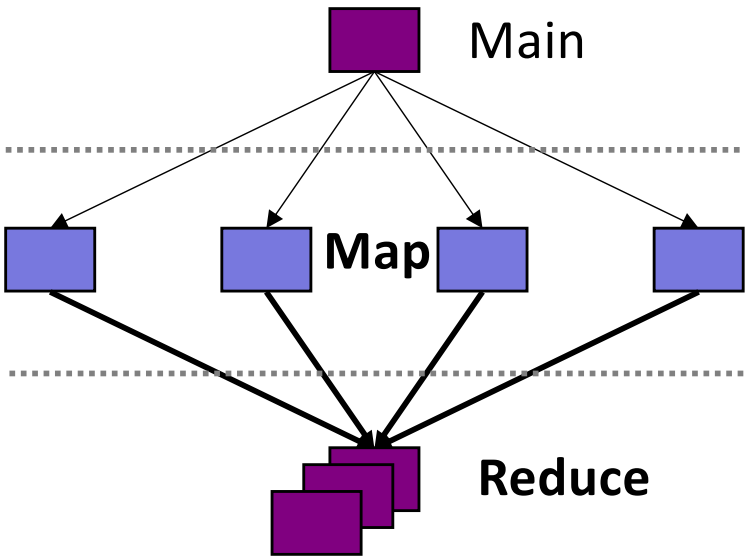


- Buffer newly inserted pairs in memory (e.g., on BigTable server)
- Periodically read, merge, and write a new SSTable file

https://www.pracdata.io/p/internal-storage-design-of-modern-33e

Big Data Workflows (e.g., MapReduce)

- For Big Data processing, multiple tasks may read the same input file.
- Read partitions, process in parallel.
  - **Parallel reads**
- E.g., MapReduce: process a stream of KV records through filter/count steps (**map**), then combine partial results (**reduce**).
- Output to another file(s).
  - **Parallel writes**



Dataflow: process data in parallel through a graph of tasks/nodes, e.g., **map** and **reduce** tasks.

MapReduce: Example

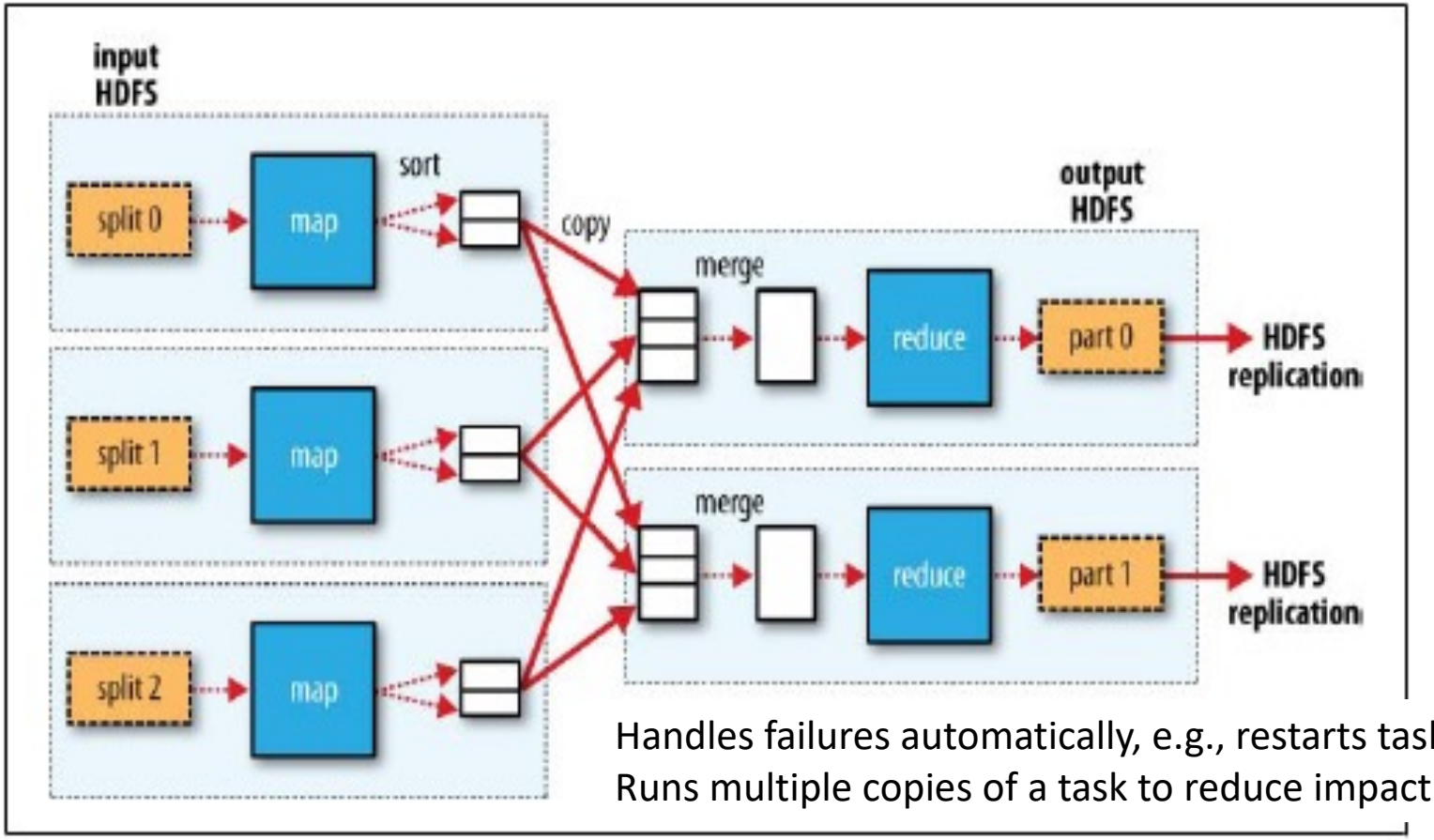


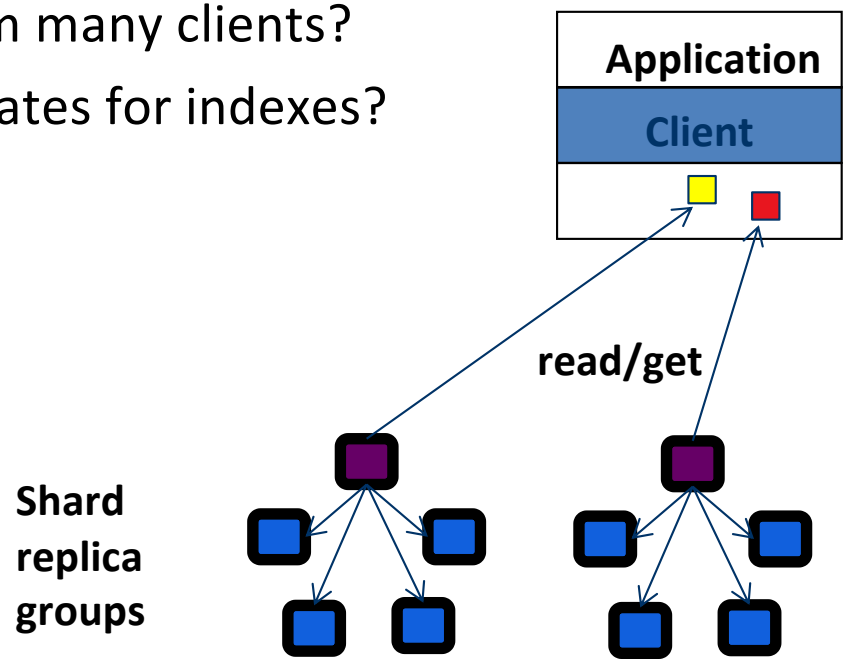
Figure 2-3. MapReduce data flow with multiple reduce tasks

HDFS ≙ GFS

Handles failures automatically, e.g., restarts tasks if a node fails;
Runs multiple copies of a task to reduce impact of stragglers.

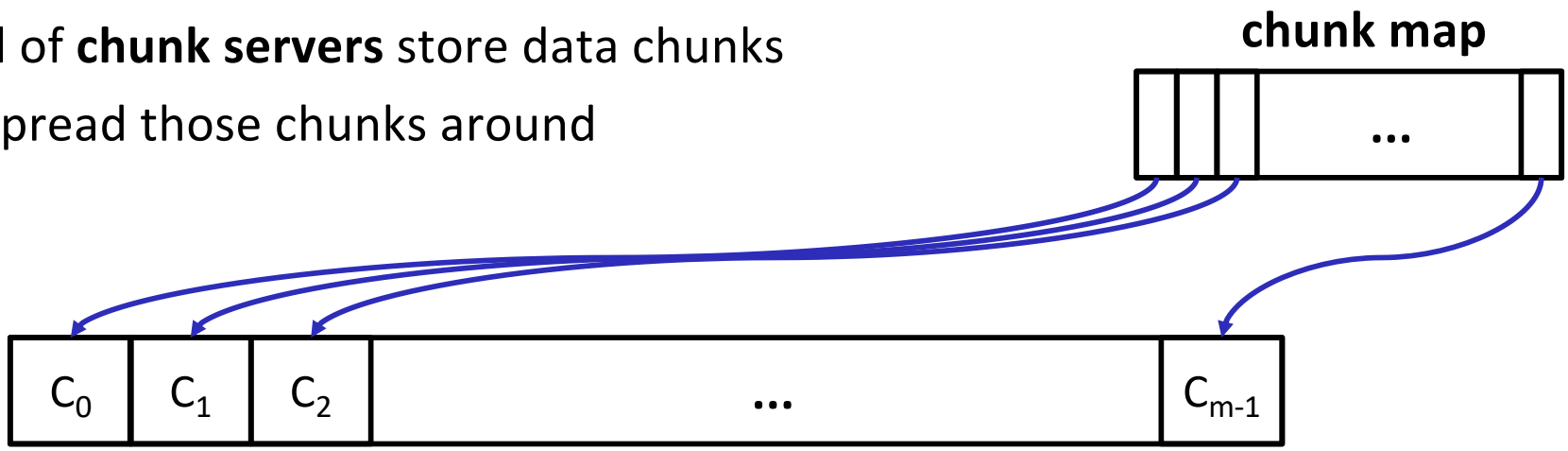
Serving BIG files?

- How to "shard" to benefit from scale?
  - Adding servers should incrementally expand capacity and bandwidth.
  - What if the active data is in a small number of very large files?
  - Parallel access to the same file from many clients?
- How to ensure atomic/consistent updates for indexes?
  - File system name space
  - Indexes for SSTable ranges
- Does our preferred structure work?
- **What changes?**



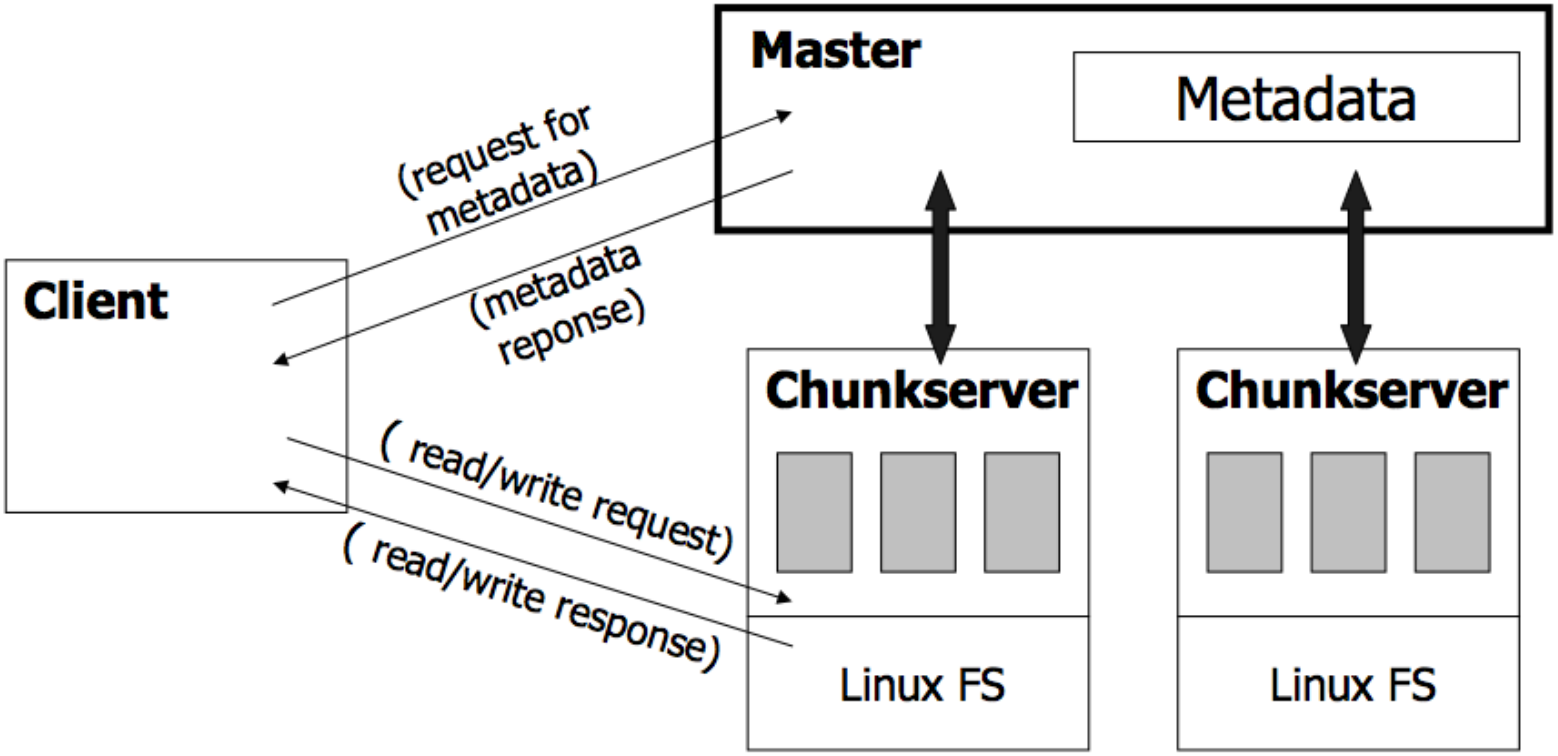
GFS architecture

- Partition file contents into fixed-size **chunks** (e.g., 64 MB)
- **Single master server**
  - Maintains name space (tree) and chunk metadata
  - Maps (filename, chunk index) to chunk locations
- Pool of **chunk servers** store data chunks
  - Spread those chunks around



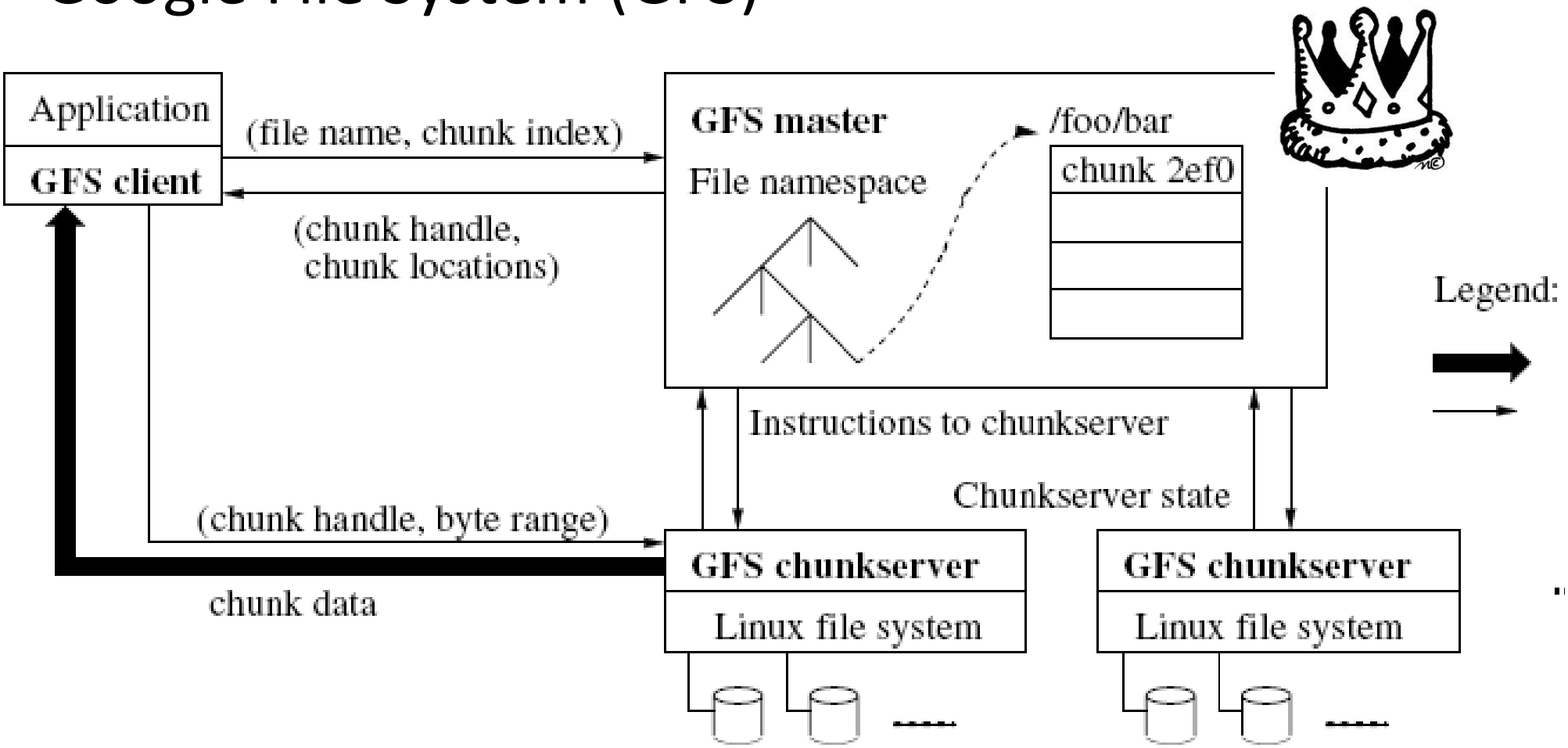
file size: m*chunk_size

GFS architecture



Separate data (chunks) from metadata (names etc.).
Centralize the metadata; spread the chunks around.

Google File System (GFS)



Really? A single master server?

- **Bottleneck**
- **Single point of failure**
- **Single point of attack**

Really? A single master server?

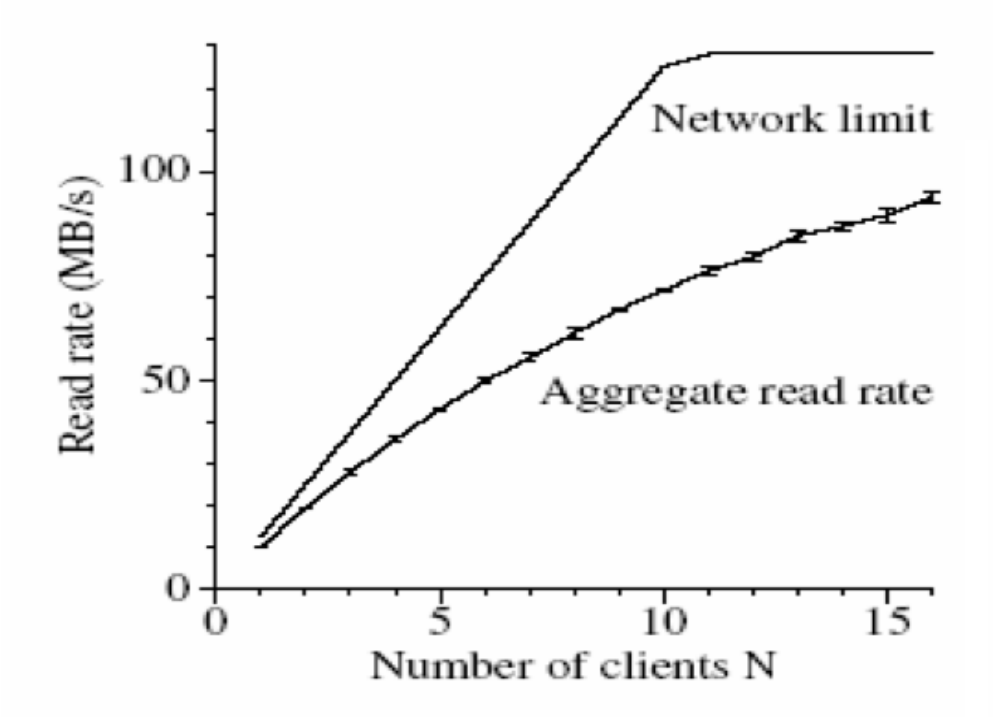
- **Bottleneck**. Keep it light and tight, memory only.
- **Single point of failure**. Log and replicate, fast recovery.
- **Single point of attack.** Protect it with a hard shell.
- **Yes, it's unconventional.** But the simplicity is compelling.
  - Can still support different file trees on different masters
  - Colossus serves multiple masters.
- GFS master keeps metadata **in memory** to max throughput.
- How much metadata? It depends. **What does it depend on?**
  - Each file has a name and attributes
  - Each chunk has a location (replicas)

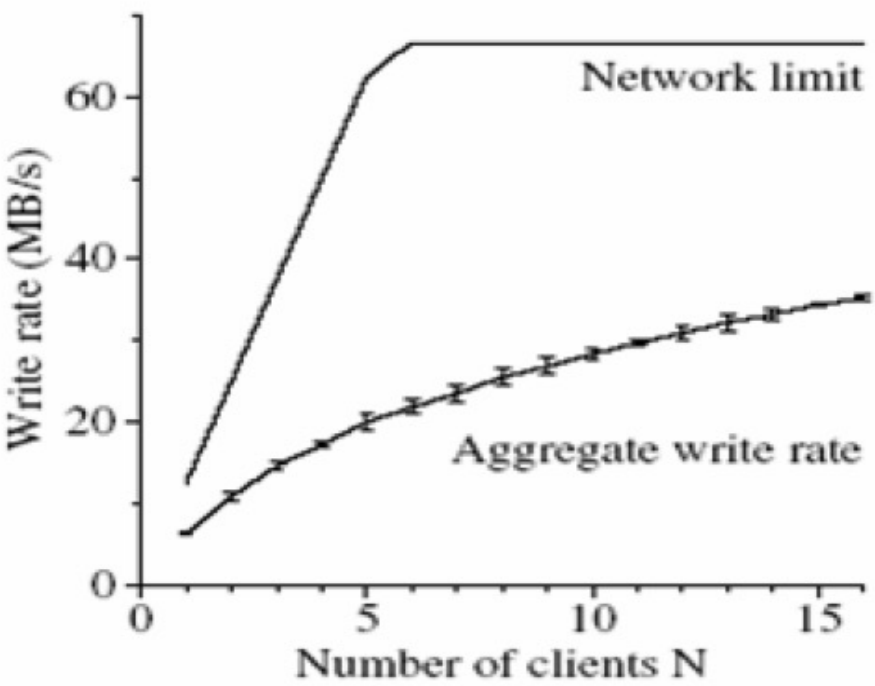
Big Idea in Data-intensive Distributed Systems

- **Separate control plane from data plane.**
- **Centralize the control plane.**

- Control traffic is for coordination and consistent behavior. A single point of control can help!
- Data traffic is where the real overheads are.
- Google principals promote this philosophy: it worked for them.
- Example: software-defined networking

GFS Scale


Network limit
Aggregate read rate
Read rate (MB/s)
100
50
0
0
5
10
15
Number of clients N


Network limit
Aggregate write rate
Write rate (MB/s)
60
40
20
0
0
5
10
15
Number of clients N

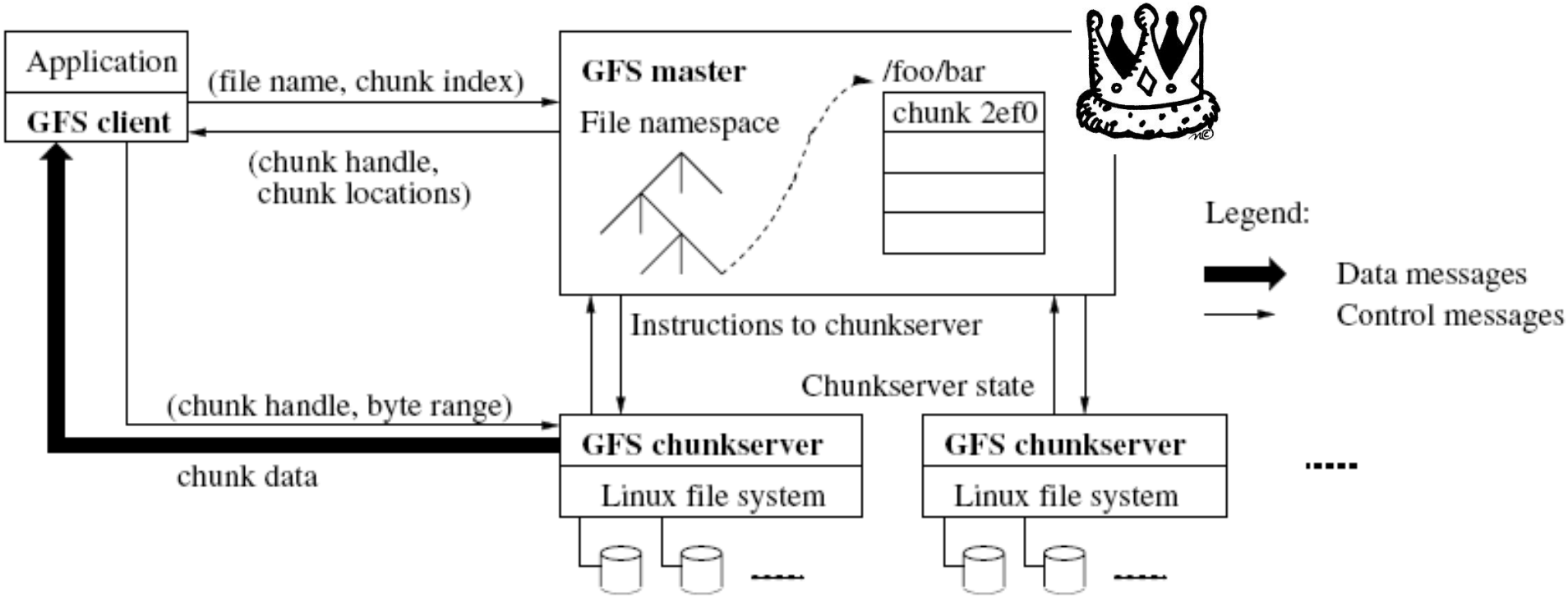
Chunks

- Variable size, up to 64MB
- Replicated on multiple nodes, e.g., x3
  - chunkserver == datanode (HDFS)
  - Stored as a local file on each replica
  - Named by a **handle**
- Master caches chunk maps
  - Per-file **chunk map**: what chunks make up a file
  - Chunk **replica map**: which nodes store each chunk

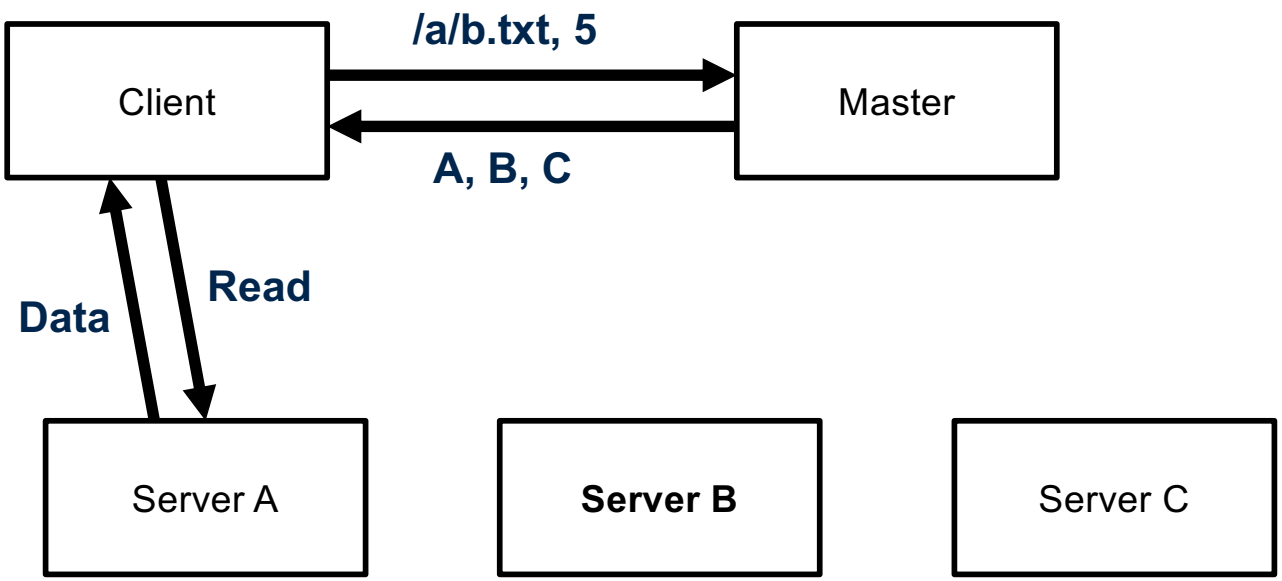
Watching the chunkservers

- They fail! They recover! Data replication rides through it.
- But master must watch failures, repair replicas, etc.
- GFS is an early paper. Let's simplify:
  - Chunkservers have session leases, so: heartbeats
  - E.g., with Chubby—BigTable uses Chubby to watch its servers
- GFS master configures replica group per-chunk
  - Spread the chunks to spread read load
  - Spread the chunk creates to spread write load
  - Replicate wisely (e.g., on different racks/switches)
  - Cassandra uses a hash distribution (like Dynamo)

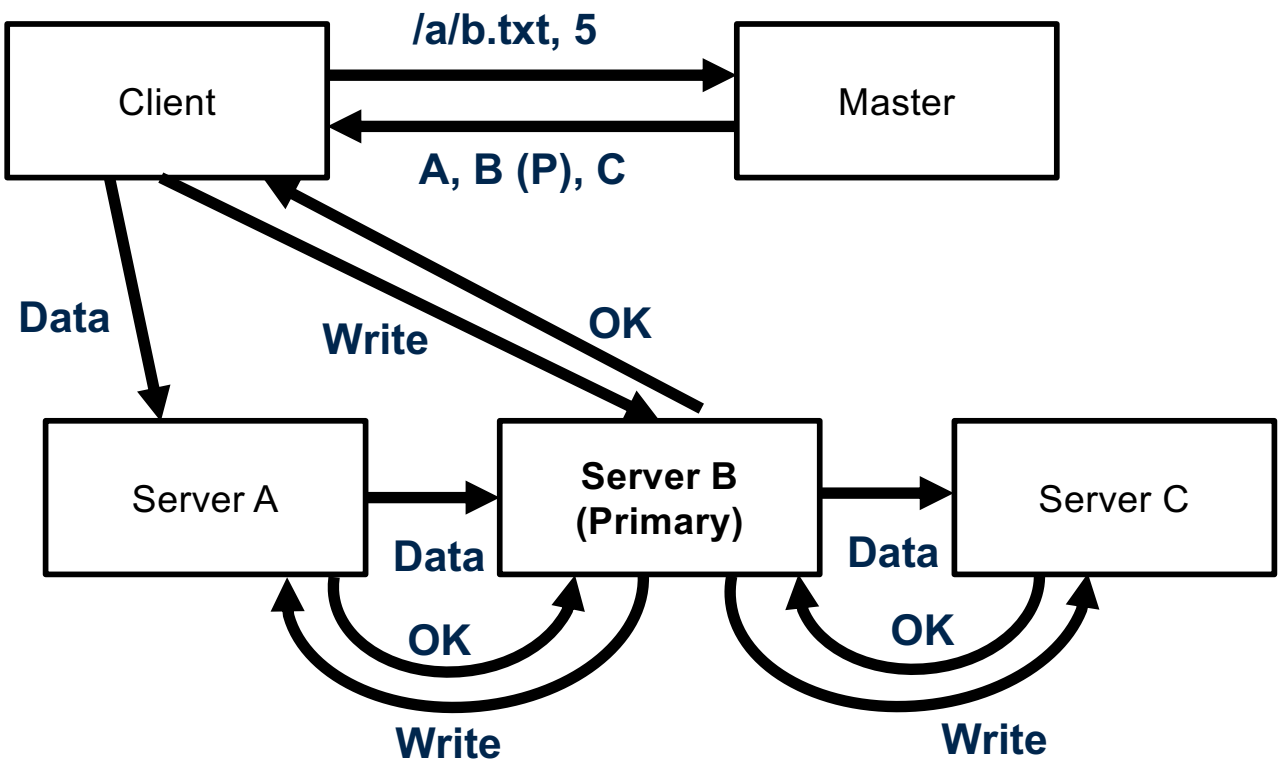
Google File System (GFS)



Read


Client
/a/b.txt, 5
Master
A, B, C
Data
Read
Server A
Server B
Server C

Write


Client
/a/b.txt, 5
Master
A, B (P), C
Data
Write
OK
Server A
Server B
(Primary)
Server C
Data
Data
OK
OK
Write
Write

GFS writes: control and data flow

- Client obtains chunk maps from master.
- Client notifies chunk primary of write.
- Clients sends data (in parallel) to closest replica.
- Replicas share the data transfer in pipelined fashion (offload client).
- Primary coordinates write.
- Primary notifies client of write completion.

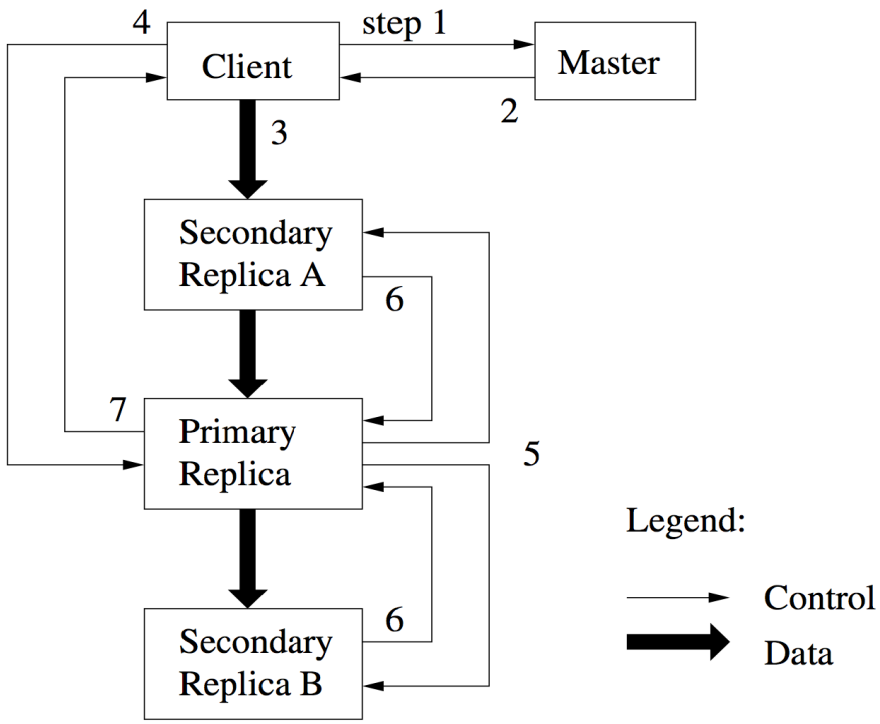


Figure 2: Write Control and Data Flow

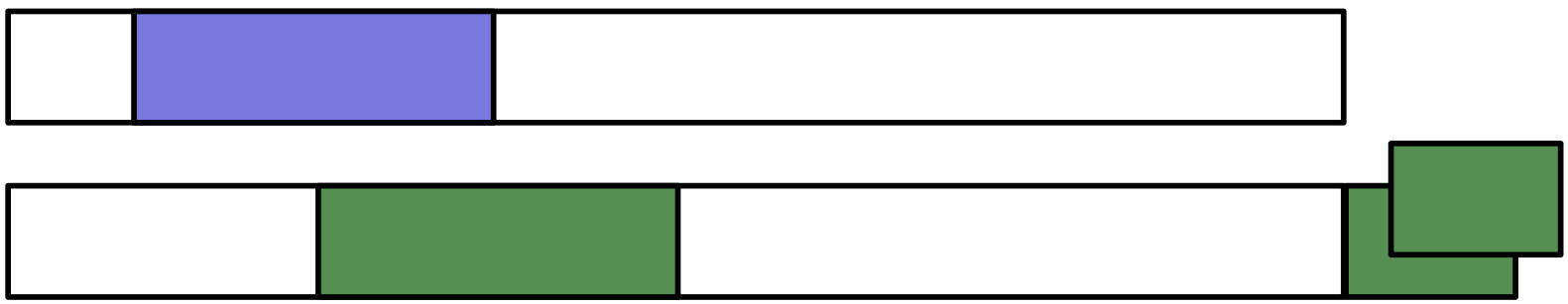
Notes on GFS write (mutation) protocol

- **Separate control flow from data flow.**
  - Minimize the cost of the data transfer.
  - Control writes for consistency.
- Client writes data to closest replica
- Replicas pass data in a chain
- Replicas hold the new data in memory and acknowledge
- Client issues write control request (no data) to primary.
- Primary sequences the write with other writes to the chunk.
- Primary issues commands to secondaries to install mutation.

Concurrent operations and consistency

All file systems have to deal with these questions.

- Concurrent read/write in overlapping ranges?
- Two processes append “at the same time”: who wins?
- On local Unix file systems, atomicity is assured.
- This atomicity is expensive for distributed file systems: it requires distributed locks and complex failure handing.

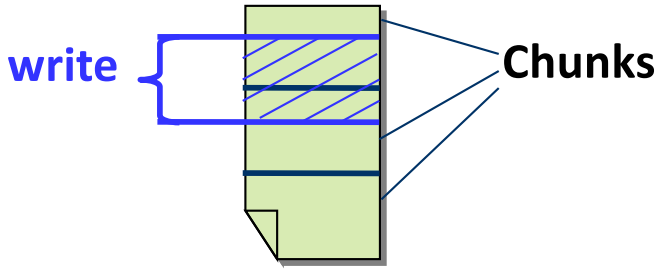


GFS: choose scalable semantics

- GFS argues that the loose semantics are "good enough".
  - "Regardless of consistency and concurrency issues, this approach has served us well."
- It helps that they control the API and applications: not stuck with Unix.
- E.g., record appends are common, and a little loose in GFS.
  - Apps: add checksums to detect partial/truncated records.
  - Apps: keep a map to detect duplicate records.

GFS consistency in a nutshell

- Concurrent access is not the common case. OK to be a little sloppy around the edges, if it helps speed and scale.
- Clients can read from any replica: a read may return stale data.
- A read may return data from an incomplete write.
- Writes that cross chunk boundaries are not guaranteed atomic.
- Record appends: hot mess



GFS: chunks and their replica maps

- Chunks are versioned.
  - **Chunk version number** passed from master to client with maps
  - Master increments it on chunkservers periodically; stale replicas lag
  - Client checks for stale chunks on read
  - Master queries/deletes stale chunks
- Cached maps have TTL consistency (stale for bounded time)
- Master can lock chunks against mutations, e.g., for snapshots

Since clients cache chunk locations, they may read from a stale replica before that information is refreshed. This window is limited by the cache entry's timeout and the next open of the file, which purges from the cache all chunk information for that file.

GFS consistency terminology

- A file region is *consistent* if all clients will always see the same data, regardless of which replicas they read from.
- A file region is *defined* after a file data mutation if it is consistent and clients will see the mutation **in its entirely**.
  - Client 1 write AAA, GFS returns successfully
  - Client 2 write BBB, GFS returns successfully
  - Can the file be ABA?

GFS consistency model

Records appended "atomically at least once".

Easy.

...but file may contain duplicates and/or padding.

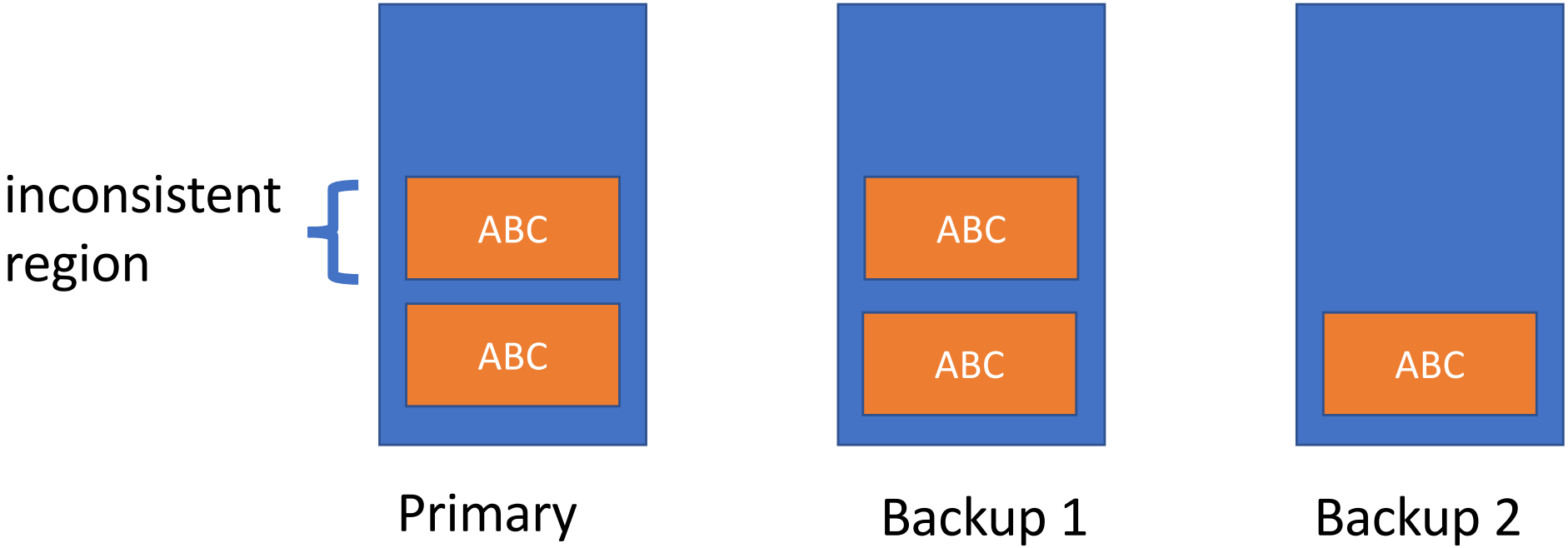
	Write	Record Append
Serial success	*defined*	*defined* interspersed with *inconsistent*
Concurrent successes	*consistent* but *undefined*	
Failure	*inconsistent*	

Writes that cross chunk boundaries are not atomic: the two chunk primaries may choose different orders.

Anything can happen if writes fail: a failed write may succeed at some replicas but not others. Reads from different replicas may return different results.

Inconsistent file region

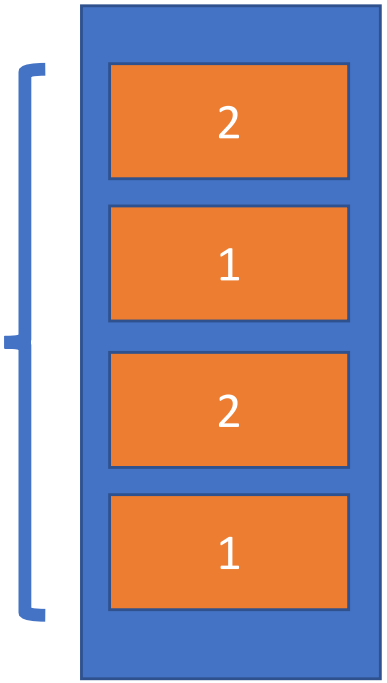
- Typically caused by failure during a write or append



Consistent but not defined file region

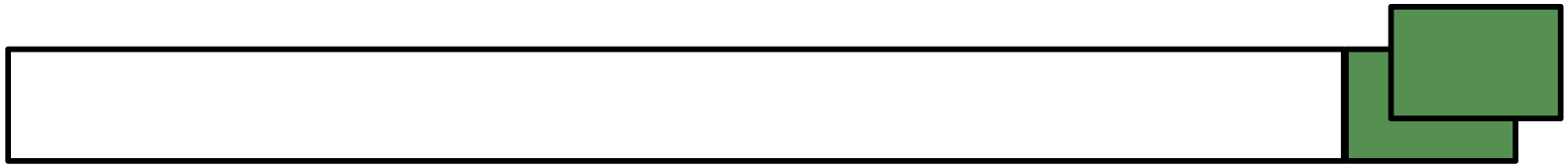
- Clients 1 and 2 simultaneously write to same multi-chunk file region
- Chunks have different primary chunkservers
- Writes end up interleaved

consistent but not defined region



Record append: “atomically at least once”

- GFS supports record appends
- If append would cross chunk boundary
  - Primary pads to chunk boundary; client retries (on next chunk).
- If append fails on any replica: client retries.
  - May cause duplicates on subset of replicas where earlier write succeeded..



"Append atomically at least once"

The primary checks to see if appending the record to the current chunk would cause the chunk to exceed the maximum size (64 MB). If so, it pads the chunk to the maximum size, tells secondaries to do the same, and replies to the client indicating that the operation should be retried on the next chunk. …

If a record append fails at any replica, the client retries the operation. As a result, replicas of the same chunk may contain different data possibly including duplicates of the same record in whole or in part. GFS does not guarantee that all replicas are bytewise identical. It only guarantees that the data is written at least once as an atomic unit. …

GFS: choose scalable semantics

Appending is far more efficient and more resilient to application failures than random writes.

In one typical use, a writer generates a file from beginning to end. It …periodically checkpoints how much has been successfully written. Checkpoints may include application-level checksums. Readers verify and process only the file region up to the last checkpoint, which is known to be in the defined state. …Checkpointing allows writers to restart incrementally...

…Moreover, as most of our files are append-only, a stale replica usually returns a premature end of chunk rather than outdated data. When a reader retries and contacts the master, it will immediately get current chunk locations.

Impact on apps

GFS applications can accommodate the relaxed consistency model with a few simple techniques already needed for other purposes: relying on appends rather than overwrites, checkpointing, and writing self-validating, self-identifying records....

Readers deal with the occasional padding and duplicates as follows. Each record prepared by the writer contains extra information like checksums so that its validity can be verified. A reader can identify and discard extra padding and record fragments using the checksums. If it cannot tolerate the occasional duplicates (e.g., if they would trigger non-idempotent operations), it can filter them out using unique identifiers in the records, which are often needed anyway…

Extra Slides

Another view of the details

GFS: master log

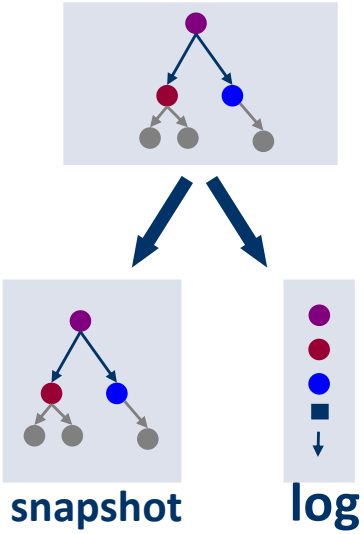
2.6.3 *Operation Log*

The operation log contains a historical record of critical metadata changes. It is central to GFS. Not only is it the only persistent record of metadata, but it also serves as a logical time line that defines the order of concurrent operations. Files and chunks, as well as their versions (see Section 4.5), are all uniquely and eternally identified by the logical times at which they were created.

Since the operation log is critical, we must store it reliably and not make changes visible to clients until metadata changes are made persistent. Otherwise, we effectively lose the whole file system or recent client operations even if the chunks themselves survive. Therefore, we replicate it on multiple remote machines and respond to a client operation only after flushing the corresponding log record to disk both locally and remotely. The master batches several log records together before flushing thereby reducing the impact of flushing and replication on overall system throughput.

The master recovers its file system state by replaying the operation log. To minimize startup time, we must keep the log small. The master checkpoints its state whenever the log grows beyond a certain size so that it can recover by loading the latest checkpoint from local disk and replaying only the limited number of log records after that. The checkpoint is in a compact B-tree like form that can be directly mapped into memory and used for namespace lookup without extra parsing. This further speeds up recovery and improves availability.

Because building a checkpoint can take a while, the master's internal state is structured in such a way that a new checkpoint can be created without delaying incoming mutations. The master switches to a new log file and creates the new checkpoint in a separate thread. The new checkpoint includes all mutations before the switch. It can be created in a minute or so for a cluster with a few million files. When completed, it is written to disk both locally and remotely.



GFS master log: details

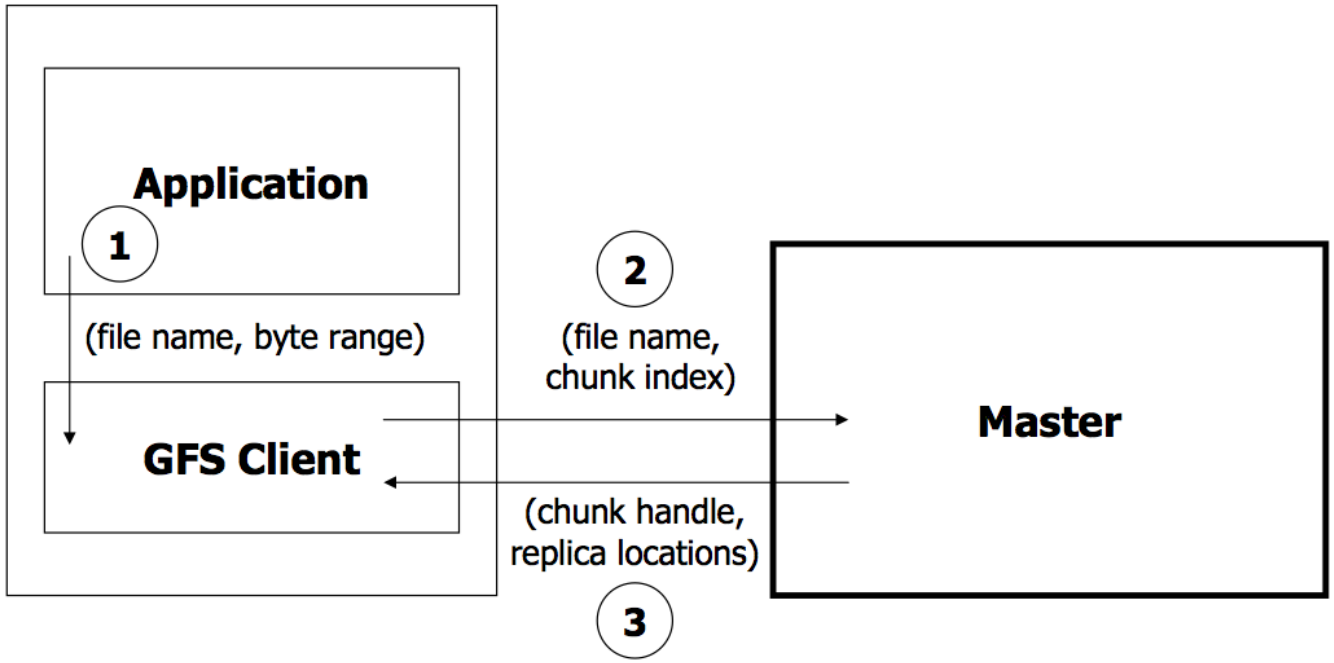
We must store [the operation log] reliably and not make changes visible to clients until metadata changes are made persistent. …Therefore, we replicate it on multiple remote machines and respond to a client operation only after flushing the corresponding log record to disk both locally and remotely.

The log is made durable by writing copies to multiple disks/machines. The log writes must complete before the master responds to the operation request. This is a form of write-ahead logging also called output commit.

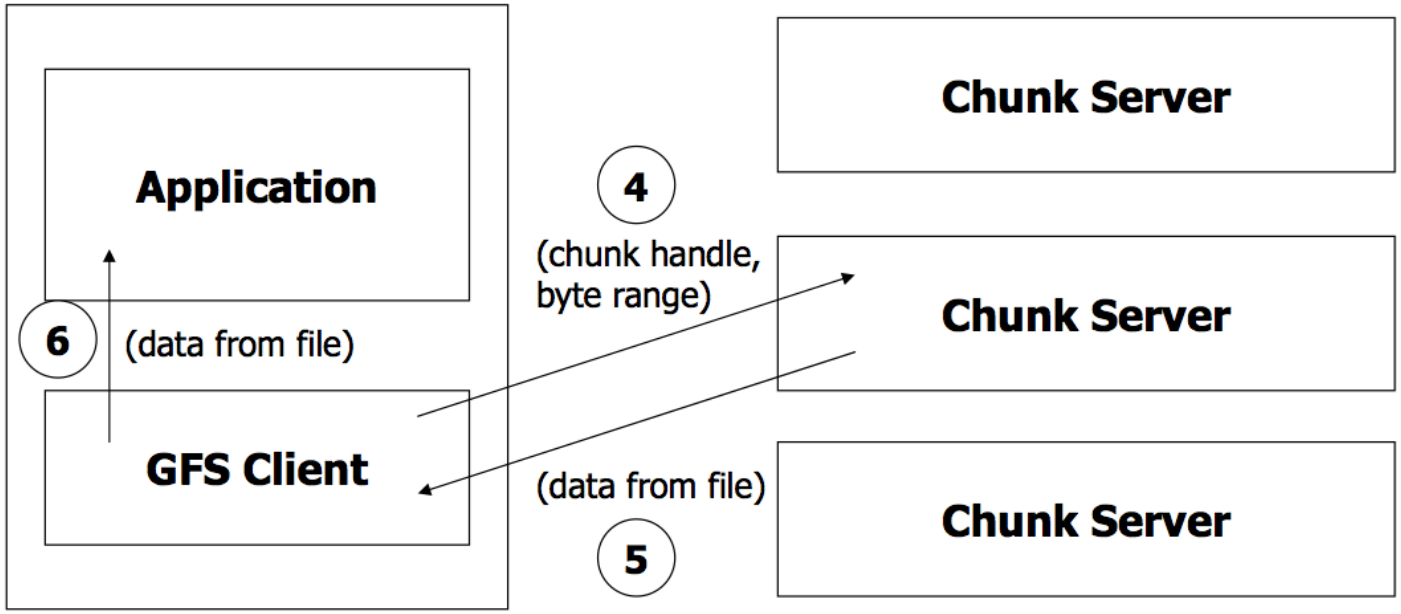
The master batches several log records together before flushing thereby reducing the impact of flushing and replication on overall system throughput.

This is a common optimization called group commit. It increases disk write bandwidth for the log at the price of increasing commit latency.

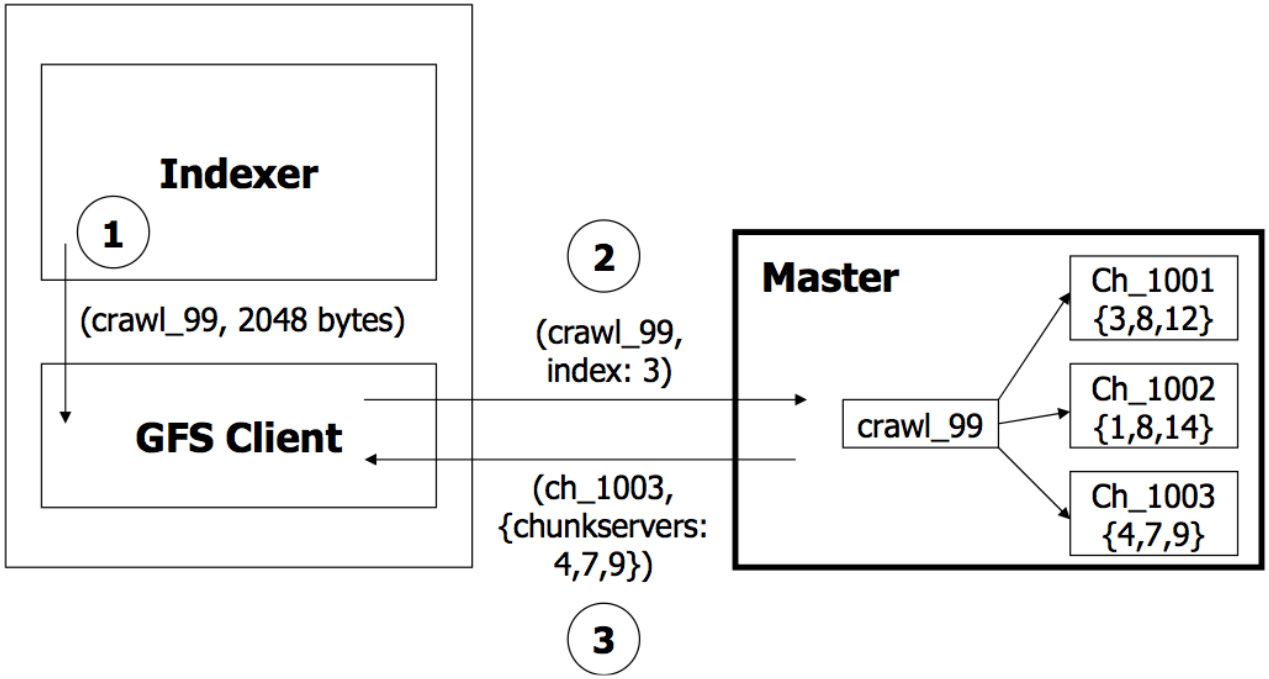
GFS Read


Application
1
(file name, byte range)
GFS Client
2
(file name,
chunk index)
Master
(chunk handle,
replica locations)
3

GFS Read

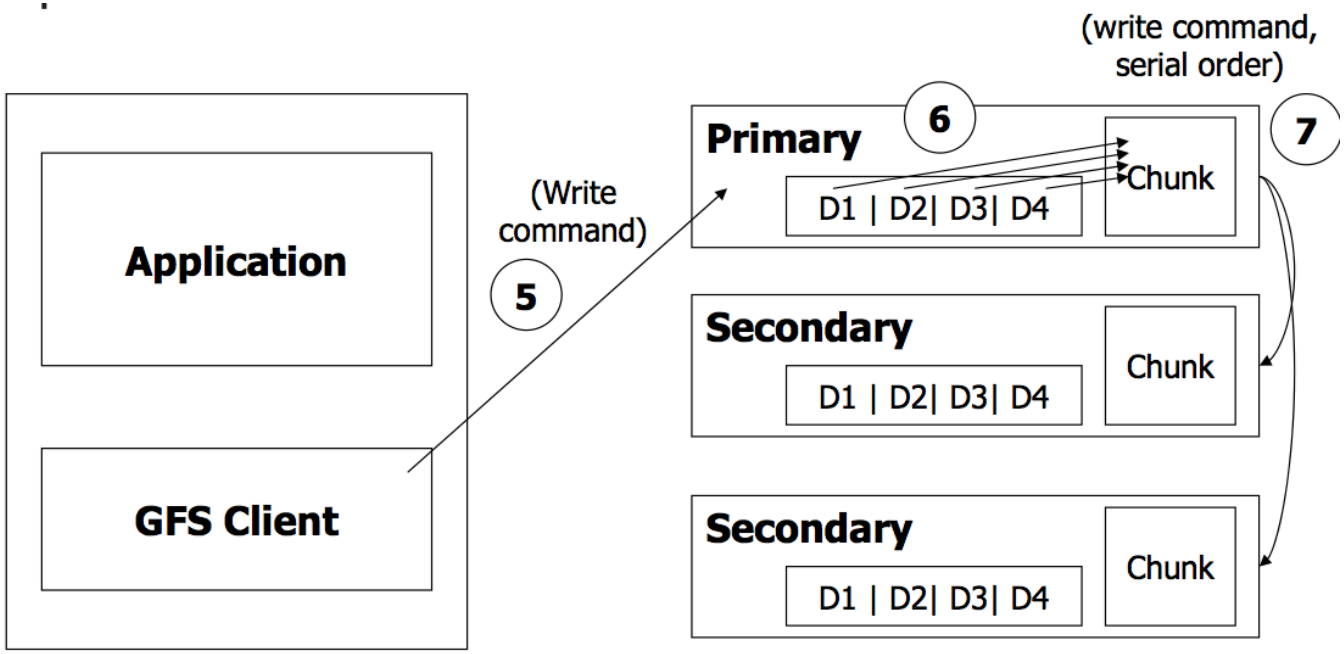

Application
GFS Client
6
(data from file)
4
(chunk handle,
byte range)
(data from file)
5
Chunk Server
Chunk Server
Chunk Server

GFS Write

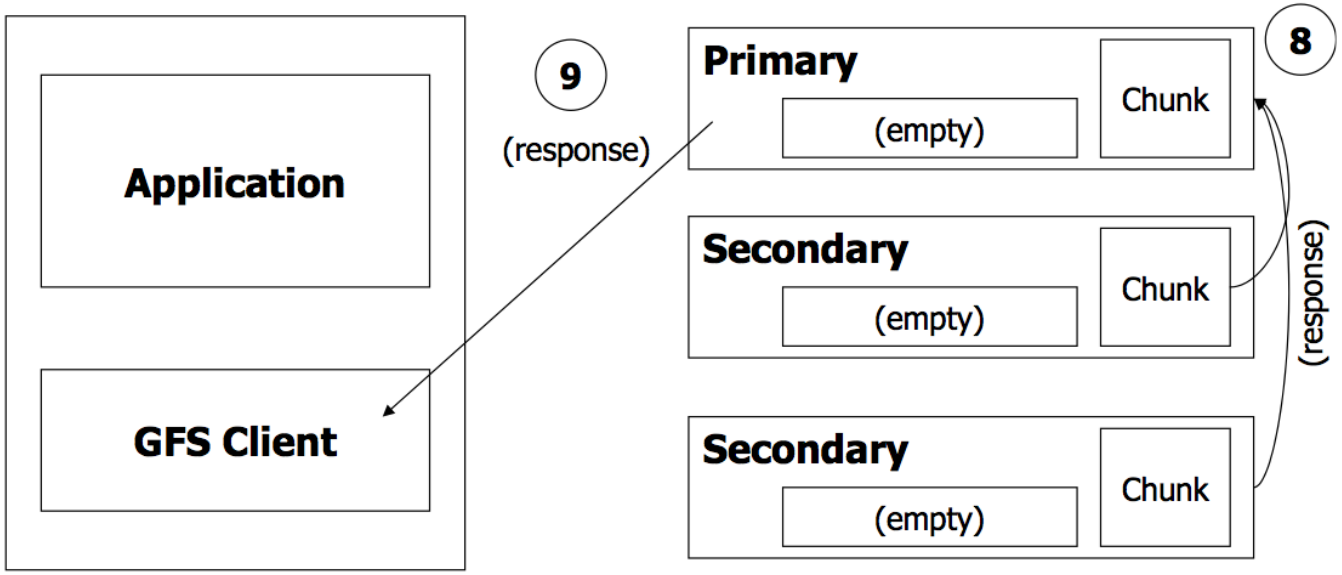


The client asks the master for a list of replicas, and which replica holds the lease to act as primary. If no one has a lease, the master grants a lease to a replica it chooses. ...The master may sometimes try to revoke a lease before it expires (e.g., when the master wants to disable mutations on a file that is being renamed).

GFS Write


(write command,
serial order)
Application
GFS Client
(Write
command)
5
Primary
6
7
D1 | D2| D3| D4
Chunk
Secondary
D1 | D2| D3| D4
Chunk
Secondary
D1 | D2| D3| D4
Chunk

GFS Write


Application
GFS Client
9
(response)
Primary
(empty)
Chunk
8
Secondary
(empty)
Chunk
(response)
Secondary
(empty)
Chunk